

SECURITY REVIEW

FOR RWA

INSIDE THIS REPORT

CONTENTS

01	AUDIT SUMMARY	03
02	METHODOLOGY	04
03	RISK CLASSIFICATION	05
04	FINDINGS SUMMARY	06
05	DETAILED FINDINGS	07
	H-01 Cross-chain Replay in borrowAsset() & swapToBorrow()	07
	H-02 First Depositor Issue in depositAsset()	08
	M-01 Missing time limit for signature	09
	M-02 Missing Approval when transferring	10
	M-03 Using transferFrom instead of transfer	11
	M-04 Use safeTransfer & safeTransferFrom	12
	I-01 Ensure Collateral Addresses and Amounts Arrays Have Matching Lengths	13
	I-02 Redundant Processed Hash Check in borrowAsset Function	14
	I-03 Gas Optimization: Revert Early if _share is Zero in removeAsset	15
	I-04 Underflow Revert: Add Check for Excess Withdrawal in removeAsset	16
	I-05 Consider adding deadline for swap	17
06	DISCLAIMER	18
07	ABOUT KANN AUDITS	19

01 - OVERVIEW

AUDIT SUMMARY

RWA engaged Kann Audits to perform a security review of the RWA smart contracts. A team of security researchers performed a manual review of the in-scope contracts. All findings identified during the assessment are documented in this report.

TOTAL FINDINGS

11

 ISSUES

CRITICAL	-
HIGH	02
MEDIUM	04
LOW	-
INFORMATIONAL	05

ENGAGEMENT DETAILS

PREPARED FOR	RWA
PREPARED BY	Kann Audits
DATE REPORTED	22 January 2025

02 - HOW WE WORK

METHODOLOGY

An engagement with Kann Audits involves a team of security researchers performing independent reviews of the codebase at the start of the engagement, followed by collaborative review sessions where auditors share findings and explore additional attack vectors to improve overall coverage.

The review included a line-by-line manual inspection of the in-scope codebase, along with analysis of the protocol architecture, access control mechanisms and state transitions. The team also assessed the code against common smart contract security best practices and industry standards.

STEP BY STEP

OUR SECURITY PROCESS

01**Kickoff & Context**

A kickoff call to understand architecture, trust boundaries, threat assumptions and intended behaviour.

02**Scope & Preparation**

Confirm the commit, files in scope, dependencies and the riskiest areas. We open a private channel between your team and our auditors.

03**AI First Scan**

Kann AI, our proprietary model built to find security bugs, scans the codebase before the manual review starts.

04**Expert Security Review**

Line-by-line analysis of exploit paths, invariant breaks, edge cases, accounting and integration risks.

05**Findings & Proofs**

Each issue is confirmed, rated and backed by a proof of concept or a precise reproduction path.

06**Mitigation Review**

We review every fix, confirm it resolves the root cause and flag partial or incorrect mitigations.

COVERAGE

THE REVIEW SPECIFICALLY CONSIDERED

- ✓ Testing against common and uncommon attack vectors
- ✓ Contract logic matching the client's intended behaviour
- ✓ Manual line-by-line review by experienced auditors
- ✓ Architecture and design analysis for secure component interactions and clear trust boundaries
- ✓ Compliance with industry best practices and known smart contract security standards
- ✓ Cross-referencing patterns with industry-standard implementations
- ✓ Access control verification
- ✓ State transitions and logic, to detect inconsistencies, unexpected behaviour or exploitation paths

RISK CLASSIFICATION

Severity combines the **impact** of an issue if exploited with the **likelihood** that the conditions for exploitation are met.

● CRITICAL	Directly exploitable with few or no preconditions, leading to theft or permanent loss of funds or loss of control of the system. Must be fixed before deployment.
● HIGH	Directly exploitable vulnerabilities that can lead to significant material loss or critical impact on the system and require immediate fixing.
● MEDIUM	Moderate loss of funds or impact on multiple users, but only under specific conditions. Exploitable without privileged access, but typically requires certain assumptions or edge cases.
● LOW	Issues with limited impact that do not put funds directly at risk, such as edge cases or deviations from documented behaviour.
● INFORMATIONAL	Non-exploitable findings such as best-practice deviations, cosmetic issues or documentation and compliance notes. They do not need urgent remediation but improve code quality and maintainability.

FINDINGS SUMMARY

● CRITICAL	● HIGH	● MEDIUM	● LOW	● INFO
-	02	04	-	05
TOTAL 11 FIXED 00 ACKNOWLEDGED 00				

ID	TITLE	SEVERITY	STATUS
H-01	Cross-chain Replay in borrowAsset() & swapToBorrow()	● HIGH	
H-02	First Depositor Issue in depositAsset()	● HIGH	
M-01	Missing time limit for signature	● MEDIUM	
M-02	Missing Approval when transferring	● MEDIUM	
M-03	Using transferFrom instead of transfer	● MEDIUM	
M-04	Use safeTransfer & safeTransferFrom	● MEDIUM	
I-01	Ensure Collateral Addresses and Amounts Arrays Have Matching Lengths	● INFORMATIONAL	
I-02	Redundant Processed Hash Check in borrowAsset Function	● INFORMATIONAL	
I-03	Gas Optimization: Revert Early if _share is Zero in removeAsset	● INFORMATIONAL	
I-04	Underflow Revert: Add Check for Excess Withdrawal in removeAsset	● INFORMATIONAL	
I-05	Consider adding deadline for swap	● INFORMATIONAL	

DETAILED FINDINGS

H-01

Cross-chain Replay in borrowAsset() & swapToBorrow()

SEVERITY

● HIGH

■ DESCRIPTION

Cross-chain Replay: If an User borrows assets on Chain A, they could reuse the same signature (generated from the same data) on Chain B, assuming the signature is valid for the same borrow hash structure on both chains.

■ PROOF OF CONCEPT

Admin signs signature with his private key with information about the collateral address, borrow address, collateral amount, borrow amount, address of the User to use signature and nonce.

Chain A User calls borrowAsset() borrows assets. Chain B User reuses the same signedHash to borrow assets.

■ MITIGATION

```
bytes32 _borrowHash = keccak256(
    abi.encode(
        collaterals,
        borrows,
        collateralAmounts,
        borrowsAmounts,
        msg.sender,
        nonce0f[msg.sender],
        block.chainid // Include the chainId in the hash to make it unique per
chain
    )
);
```

H-02

First Depositor Issue in depositAsset()

SEVERITY

● HIGH

■ DESCRIPTION

The depositAsset function is vulnerable to the first depositor bug, where the first depositor intentionally gain an unfair advantage due to how shares (`_share`) are calculated for initial deposits.

Example: Bob wants to deposit 10 WETH Alice sees Bob's transaction in the mempool and frontruns it (Alice becomes the first depositor). She deposits just 1 wei. Bob deposits the 10 WETH but due to rounding issues he receives 0 shares. Alice calls `removeAsset()` and gets 10 WETH + 1 wei withdrawn to her since he owns the 100% of the shares.

M-01

Missing time limit for signature

SEVERITY

● MEDIUM

■ DESCRIPTION

Some actions in financial contracts are time-sensitive, and allowing a signature to be valid indefinitely could break expected business rules. For instance, if the contract terms change or the collateral value fluctuates, an old signature might lead to unfair borrowing conditions or outdated agreements.

■ PROOF OF CONCEPT

Example: A user signs a borrowing agreement for certain collateral amounts and borrowing amounts today. Later, the market conditions change, and collateral values drop or borrowing values gets too high, making the original agreement unfair for the contract owner. By adding a time limit (e.g., valid for 1 hour), the signature would become invalid once the time expires, ensuring that the agreement can only be executed within the intended window

■ MITIGATION

```
bytes32 _borrowHash = keccak256(
    abi.encode(
        collaterals,
        borrows,
        collateralAmounts,
        borrowsAmounts,
        msg.sender,
        nonceOf[msg.sender],
        deadline // Add expiry
    )
);

// Check if the signature is expired
if (block.timestamp > deadline) {
    revert SignatureExpired();
}
```

M-02

Missing Approval when transferring

SEVERITY

● MEDIUM

■ DESCRIPTION

In `borrowAsset()` when user is trying to send collateral it will fail since there is missing approval. Same applies for in `repayLoan()`.

■ MITIGATION

Add approve for contract to use the tokens of the user.

M-03

Using transferFrom instead of transfer

SEVERITY

● MEDIUM

■ DESCRIPTION

In `borrowAssets()` when contract is trying to send borrows to user it incorrectly uses `transferFrom` instead of `transfer` when transferring borrow tokens from the contract to the user. This implementation error will cause the function to revert due to missing allowances. same applies for in `repayLoan()` when trying to send collateral back to user.

■ MITIGATION

```
token.safeTransfer(msg.sender, _amount); //use this when transferring to user.
```

M-04

Use safeTransfer & safeTransferFrom

SEVERITY

● MEDIUM

■ DESCRIPTION

Some ERC-20 tokens will return on failure instead of reverting a transaction, Some tokens will even not return any value. not all tokens adhere to the standart of ERC-20

■ MITIGATION

Consider adding OpenZeppelin SafeERC20.

I-01

Ensure Collateral Addresses and Amounts Arrays Have Matching Lengths

SEVERITY

● INFORMATIONAL

■ DESCRIPTION

The lengths of the collaterals and collateralAmounts arrays are not explicitly checked within the function. If their lengths differ, this can lead to mismatched indexing, potentially causing unexpected behavior or errors when processing collateral addresses and their corresponding amounts.

This issue assumes the off-chain logic ensures both arrays are correctly populated. However, without a validation check in the contract, there's no guarantee of alignment, which could result in incorrect collateral handling.

■ MITIGATION

```
if (collaterals.length ≠ collateralAmounts.length) revert
```

I-02

Redundant Processed Hash Check in borrowAsset Function

SEVERITY

● INFORMATIONAL

■ DESCRIPTION

Summary: The processed[_borrowHash] check in the borrowAsset function is redundant because the nonceOf[msg.sender] mechanism already prevents signature reuse. Reusing a signature with an outdated nonce will fail the isAdmin verification, as the recomputed _borrowHash will not match the original signed hash.

■ MITIGATION

Consider removing the processed[_borrowHash] mapping and its associated logic to simplify the code and reduce gas costs.

I-03

Gas Optimization: Revert Early if _share is Zero in removeAsset

SEVERITY

● INFORMATIONAL

■ DESCRIPTION

In the removeAsset function, if the _share of msg.sender is zero, the function still proceeds to execute and consumes unnecessary gas before returning. Instead of allowing the transaction to continue and waste gas, it would be more efficient to revert the transaction early when _share is found to be zero. This prevents further execution and optimizes gas usage.

■ MITIGATION

```
if(_share == 0) revert
```

I-04

Underflow Revert: Add Check for Excess Withdrawal in removeAsset

SEVERITY

● INFORMATIONAL

■ DESCRIPTION

In the removeAsset function, if a user attempts to withdraw more shares than they hold, the transaction will fail, but the reason for failure may not be immediately clear to the user. This is due to the absence of a check to verify if shareToWithdraw exceeds the balance of shareOf[msg.sender][token]. Since shareOf[address]

■ TOKEN

is a uint256 and cannot be negative (with a minimum value of 0), the transaction fails when the withdrawal amount is too large, but the failure message is not informative. Adding a condition to explicitly check and revert the transaction with a meaningful message would improve clarity and user experience.

■ MITIGATION

```
if (shareToWithdraw > shareOf[msg.sender][token]) revert
```


I-05

Consider adding deadline for swap

SEVERITY

● INFORMATIONAL

■ DESCRIPTION

In the Mooniswap swap function, users can specify a minAmount parameter to protect against slippage, ensuring that the output of the trade meets a minimum threshold. However, the function does not include a deadline parameter, which means users can set a high minAmount and wait indefinitely for the market to meet favorable conditions. This behavior could lead to inefficiencies in the protocol, as transactions could be left pending for extended periods without completing.

■ MITIGATION

Consider adding deadline for swap

DISCLAIMER

A security audit can never guarantee the complete absence of vulnerabilities. Audits are a time, resource and expertise-bound effort in which we aim to identify as many issues as possible.

This review covers only the files listed in the Scope section at the audited commit. Code changed after that commit, external dependencies, deployment configuration and operational practices may introduce additional risk.

ABOUT KANN AUDITS

Kann Audits is a Web3 security audit company helping DeFi protocols, blockchain networks, and infrastructure teams build securely at every stage, from development to launch and beyond.



EXPERT SECURITY AUDITS

Human-led smart contract security audits built around the architecture, business logic, integrations and risk profile of complex Web3 protocols.



AI SECURITY SCAN

Kann AI extracts Solidity call graphs and returns function-level security signals for expert validation.



FORMAL VERIFICATION

Define security properties for critical protocol behaviour and evaluate whether the implementation satisfies those specifications.



INCIDENT RESPONSE

A direct triage path for suspected exploits, critical vulnerabilities, suspicious transactions and urgent remediation review.



KANN AUDITS
WEB3 SECURITY AUDITS

GET IN TOUCH

WEBSITE	kannaudits.com
PORTFOLIO	github.com/KannAudits
EMAIL	contact@kannaudits.com
X	@KannAudits
TELEGRAM	@KannAudits